DEEP RESEARCH / SOURCE-LEVEL REVIEW

DeepSeek Harness（DSH）深度剖析

从 Cordis、事件溯源与 PTC，到安全边界、性能证据和采用决策

研究基线	dsh-v0.1.2-alpha.2 / commit 0a53fb55
研究日期	2026-08-31
证据范围	官方源码、设计文档、论文、发布说明与独立安全研究
核心判断	架构样本价值高；生产安全与兼容性成熟度仍不足

面向 Agent 架构师、平台工程师与技术决策者

执行摘要

一句话结论： DSH 是一套可重组、可回放的 Agent runtime 内核与参考发行版，架构创新强；但截至 alpha.2，API / session 兼容性、安全隔离和端到端 benchmark 证据仍不足，不应在缺少外层容器隔离时直接承载不可信工作负载。

维度	判断	原因
架构原创性	强	Cordis 时空可组合性 + capability seam + event-sourced session
扩展能力	强	模型、loop、tool、skill、sandbox、storage、UI 均可替换
轨迹与审计	强	模型可见内容与 request header 可从同一日志重建
工程可维护性	中	门禁与文档优秀，但 258 packages / 86-row base 组合带来高认知成本
生产安全	弱	官方未审计；same-world file sandbox；PTC / workflow 不是 OS 隔离
兼容稳定性	弱	developer preview；破坏性变更；旧 session 无迁移链
性能证据	中-弱	有认真优化机制，但缺少可解释的公开端到端 benchmark

优先结论

- 把 DSH 当作 Agent runtime 架构样本与插件平台，而不是单纯的 coding agent UI。
- 任何比较都必须固定 model / preset / tool mode / permission policy / commit，否则结果没有解释力。
- 首次试点使用 native + workspace-write + ask，并放进 disposable container / VM；PTC 单独隔离。
- 只有任务收益、安全边界和升级成本三项同时过线，才进入有限生产。

目录

执行摘要 2

优先结论 . . . 2

目录 3

直接结论 . . . 5

1. 它到底是什么 . . . 5

2. 源码规模与工程信号 . . . 5

3. Cordis：DSH 的真正内核 . . . 6

4. Agent loop：一个 turn 不是一次模型调用 . . . 7

5. Session event sourcing：强项与债务 . . . 7

6. 工具体系与 PTC . . . 8

7. 子 Agent、Workflow、Skills 与 MCP . . . 8

8. 安全：正确的威胁模型 . . . 9

8.1 已有控制 . . . 9

8.2 必须明确的边界 . . . 9

8.3 间接 Prompt Injection . . . 9

8.4 旧审计报告如何解读 . . . 9

9. 性能：有哪些证据，哪些只是推断 . . . 9

10. 与相邻方案的定位 . . . 10

11. 采用建议 . . . 11

立即值得试 . . . 11

暂缓生产 . . . 11

推荐的 30 天验证 . . . 11

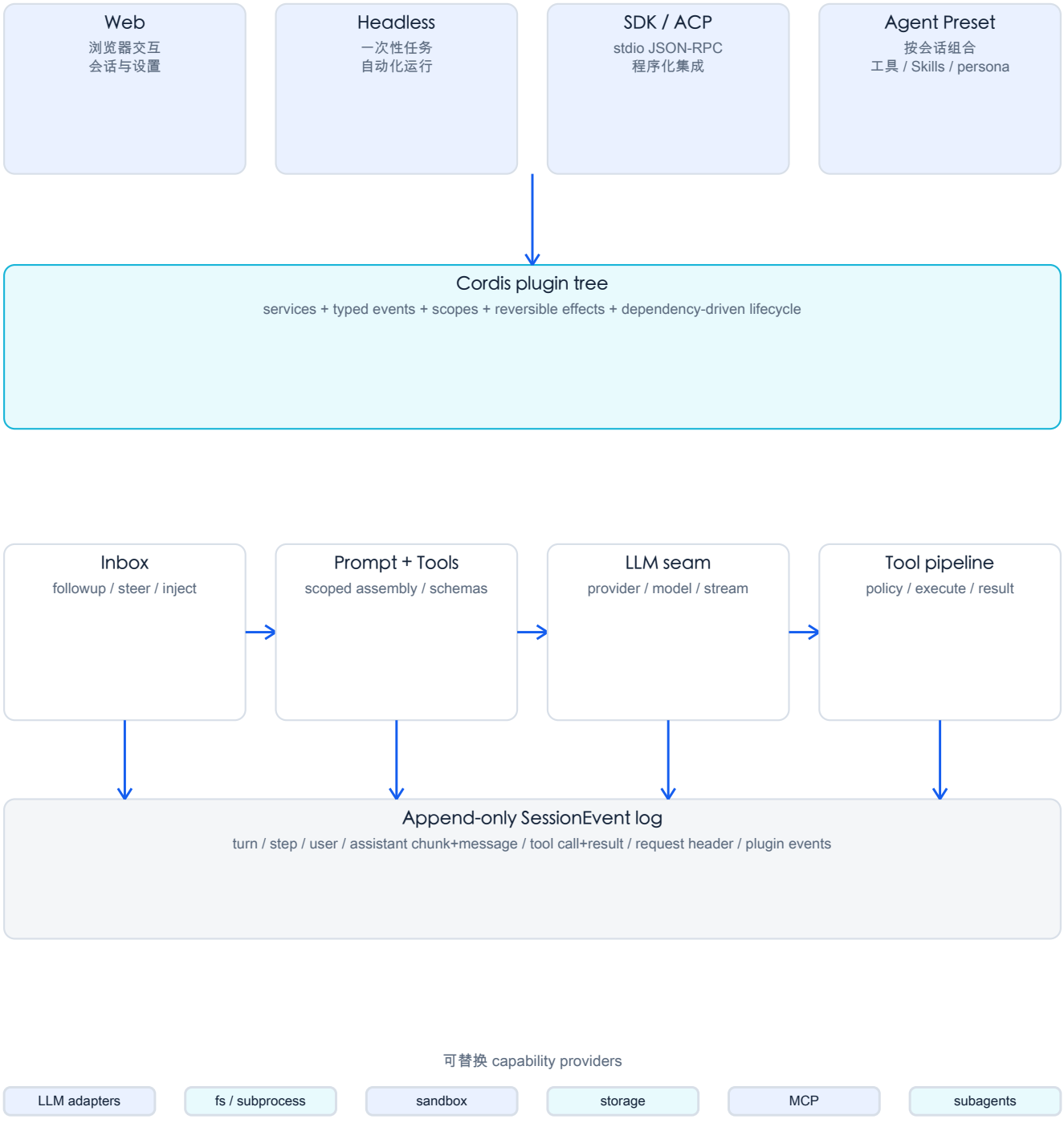
12. 最终判断 . . . 12

资料与证据账本 . . . 12

研究限制 . . . 12

DSH 运行时：插件树包围一条可重建的执行脊柱

图中蓝色箭头表示在线控制流；底部事件流是恢复、回放、分叉、UI 与审计的共同事实源。



受众：具备 LLM / Agent 工程背景的架构师、平台工程师和技术决策者

研究日期：2026-08-31

源码基线：`dsh-v0.1.2-alpha.2`，commit `0a53fb55bea101816fa226bb964ae2bed71c343b`

范围：架构、运行时、会话与工具模型、PTC、生态、安全、性能证据、竞争定位和采用建议。

排除：不评测 DeepSeek 模型本身，不把 GitHub 热度当作技术正确性的证据，也不把旧 rc 版本漏洞未经核验地外推到当前版本。

直接结论

DSH 不是“DeepSeek 版 Claude Code”，也不是一个推理框架。它更接近一套可重组的 Agent 操作系统内核 + 官方参考发行版：Agent loop、模型适配器、工具、Skills、会话、存储、沙箱、子 Agent、调度和 UI 都是 Cordis 插件。官方对它的定义是开源 agent harness，当前仍是 developer preview，核心 API 会继续发生破坏性变化。官方发布页 · GitHub README

它最有价值的创新不是工具数量，而是三件事：

1. 时空可组合性：插件声明依赖（空间），注册的副作用可逆（时间），从而支持依赖驱动的挂载、卸载、替换和 HMR。Cordis 论文把这分别形式化为 reactive coeffects 与 revertible effects。Cordis 论文
2. “模型可见即日志化”：模型输入不是散落在进程内的隐式状态，而是从 append-only SessionEvent log 与 request header 重建。恢复、分叉、回放、UI 和遥测共享同一事实源。架构文档
3. 能力 seam：定义、提供者、消费者三者分离。替换一个 filesystem/subprocess provider，可以把 Bash、PTY、LSP 一起迁到远端执行世界，而不是复制整套 Agent loop。能力架构

但它目前不是生产默认选项：版本是 alpha，官方安全声明明确称未经过安全审计、不可视为 production-ready；默认沙箱只约束文件副作用，不隔离网络和进程；PTC 的 worker-thread `run_code` 只是资源 containment，不是 OS security boundary；插件本质上是宿主进程中的受信代码。安全声明 · PTC runtime 边界

采用判断：适合做 Agent 平台研究、插件化能力平台、可回放实验台和内部原型；暂不适合未经额外容器隔离就处理不可信仓库/网页/插件，也不适合要求 API 稳定、会话长期可迁移、正式 SLA 的企业生产控制面。

1. 它到底是什么

“Harness”是模型与真实环境之间的运行层。模型负责推理；Harness 负责环境理解、工具调用、状态持续、权限、故障恢复和协作。DSH 将这些能力全部插件化，并提供 Web、headless、SDK、sdk-minimal 与 ACP 等运行 profile。Web 是交互产品面，headless 是一次性任务，SDK 通过 stdio JSON-RPC 服务程序化调用，Python SDK 打包同版本运行时。官方定位 · Python SDK

它也不被 DeepSeek 模型锁死。官方 DeepSeek adapter 之外，`dsh-llm-pi-ai` 支持 Anthropic、OpenAI、Bedrock、Vertex、Azure、Codex 以及自定义 OpenAI-compatible provider；会话在首次请求后固定自己的模型路由，以保证回放语义。模型配置

当前成熟度必须被写进任何技术选型：`0.1.2-alpha.2` 是 2026-08-30 的 pre-release；README 明示 developer preview 与 compatibility-breaking changes。发布说明

2. 源码规模与工程信号

对上述 commit 的源码快照统计：`packages/` 下有 258 个 `package.json`、2,965 个 TypeScript/TSX 文件、981 个 `*.spec/test.ts(x)`，基础 bundle 本身组合 86 个 Cordis row；同时有 303 份中文 package README。这个统计说明两件相反的事：测试与文档纪律非常强；同时包粒度极细、认知成本和组合调试成本也很高。

工程门禁覆盖 typecheck、lint、Vitest、覆盖率、快照、Web 性能/压力测试、Node 兼容、文档链接、插件/模块图、配置目录、包不变量和发布验证。源码大量使用判别联合、声明合并、不可变快照、显式错误码和运行时 invariant。这些是“平台内核”而非 demo 的信号。

反面是：极细包化会把简单调用变成跨 service、event、scope、projection、persistence 的多跳路径；插件激活又由依赖可用性驱动，调试者需要同时理解 Cordis fiber 生命周期和产品领域语义。对于只想快速改一个 coding agent prompt/tool 的团队，这是一种过度架构。

3. Cordis：DSH 的真正内核

Cordis 的上下文 `ctx` 同时是服务仓库、事件总线和副作用生命周期边界：

- 插件通过 `inject` 声明所需服务；依赖未就绪时插件处于 PENDING，依赖消失时自动卸载，恢复后重新挂载。
- `ctx.effect()` / `ctx.on()` 注册可逆副作用；插件卸载时 disposer 逆向撤销，减少 HMR 后残留 listener、tool 或 provider。
- 事件按语义分为 `emit`、`waterfall`、`parallel`、`serial`、`bail`。其中 waterfall 是 around-middleware：listener 必须调用 `next()` 才会继续，否则会短路。
- profile 是可命名的发行组合，bundle 是 Cordis rows 的分发层，后续 patch 按 id 替换完整 config；Web 可 live reload，stdio/one-shot profile 则在启动时一次性应用，避免运行中替换依赖破坏生命周期。

Cordis primer · 插件生命周期

这是 DSH 的差异化，也是它最大的学习门槛。它把“扩展点”提升为一致的系统模型，但 waterfall 的短路、动态依赖重装、patch 整体替换和跨 scope shadowing 都可能形成非局部行为。插件作者需要把 disposer、排序和失败模式当作一等设计，而不能只会注册一个 callback。

4. Agent loop：一个 turn 不是一次模型调用

DSH 的默认 loop 是 ReAct 变体，但边界定义更精确：

- turn：从一个用户/协调输入开始，包含零到多个 step，直到不再欠模型请求或工具后续。
- step：一次模型请求 + 该响应中所有工具调用的调度与结果。
- inbox：`next-turn` 与 `next-step` 两条有序队列；followup、steer、inject 的差异在于投递边界和是否唤醒 driver。

主路径是：`turn/start` → `claim` → `pre-step` → `step/start` → `user/message` → `request/header` → `LLM stream` → `assistant/chunk*` → `assistant/message` → `tool/call*` → `tool pipeline` → `tool/result*` → `step/end` → `turn/end`。所有 turn/step/message/tool 事实都写入 session log，实时拦截点则走 `agent/*` 或 capability events。生命周期

值得肯定的细节：

- 原始 stream chunk 被保留，既能复现 UI，也能形成最终 assistant message。
- 工具并行不是“全部 Promise.all”：工具声明 `isConcurrencySafe` 才进入有界 rolling pool；exclusive call 形成 barrier；dispatch 可以重叠，但结果和附加上下文按模型原始顺序提交。
- max-token、abort、scheduler failure、tool skip 都有不同的 durable 语义，取消时未启动的工具也会产生合成错误结果，保持回放结构完整。
- LLM request 被 deep-freeze，并由 invariant 在 dispatch 时重新从 session log 推导比对，防止“实际发给模型的内容”和日志分叉。

这套设计更重，但可审计性明显优于把 messages、UI state、tool state 各自维护的常见实现。

5. Session event sourcing：强项与债务

Session 是 append-only typed event log；LLM history 通过 `deriveMessages()` 从 surface 节点投影，不单独存储。普通追加采用增量缓存，复杂度近似 O(新增节点)；compaction 的 replace 会提升 generation 并触发重建。Request header 记录 provider/model、adapter defaults、system prompt 与 tool schema 快照，因此历史消息和请求 envelope 都可重建。Session 文档

默认持久化是 JSONL；另有 SQLite provider。事件先同步进入内存，再在固定批窗口后台刷盘，`session/flush` 是下一普通 turn 前的耐久性屏障。崩溃时若发现未闭合 turn，不截断日志，而是追加 synthetic `interrupted` 结尾，保留已经耐久化的长任务记录。持久化文档

代价同样明显：

- session format 不提供旧版本迁移链；当前构建遇到更旧或更新格式会拒绝加载。这对 alpha 期合理，对长期企业留存却是硬风险。
- 插件可以通过 declaration merging 增加 event type；如果缺少相应插件，未知且非 `ignorable` 的事件会拒绝会话加载。可组合性和可移植性之间存在张力。
- Event sourcing 解决“事实可回放”，不自动解决外部副作用的 exactly-once。Bash、网络提交、第三方 MCP 行为仍需调用者提供幂等性和补偿。

6. 工具体系与 PTC

工具由 schema、执行函数、输出 schema/render、并发分类、timeout 和 UI presenter 组成。执行经过 `tools/pre-execute → monotonic guards → tools/execute → tools/post-execute → finalizeContent → tools/result`。模型只看到显式 allowlist 的 name/description/parameters，host-only callback 不会泄露到 wire。原始 JSON Schema 工具（包括 MCP）也走同一 registry。工具体系

PTC（Programmatic Tool Calling）有三种呈现模式：

- `native`：模型看到每个工具 schema（默认）。
- `ptc`：模型只看到 `run_code` 与生成的 TypeScript SDK。
- `both`：两者都看到。

PTC 的收益不是“更强的模型”，而是把循环、分支、fan-out 和中间结果过滤移进一次本地程序：多个工具调用可以少经历模型 round trip；中间值不必全部重新进入上下文；稳定 SDK 前缀有利于 provider cache。内部 binding 调用仍重新进入完整 tool policy pipeline，且保留关联事件。

但 DSH 没有发布 DSH-specific 的端到端质量/成本 benchmark，不能把设计动机当作实测收益。PTC 还引入新的失败面：模型必须生成可执行、可终止且只返回 lossless JSON 的程序；中间 binding 值没有字节上限；只有外层输出受 64 MiB 默认 cap；`worker.terminate()` 不会清理模型代码自行创建的 OS 子进程。PTC runtime

最关键的安全事实：当前 worker-thread backend 明确是 containment, not a security boundary。它清空环境、限制 heap/CPU/wall/output、每次新建 worker，但模型程序仍能访问 Node APIs，权限近似 Bash。文件沙箱不会自动包住 worker 线程。因此 PTC 只应在可信代码/可信内容或外层容器中使用。

7. 子 Agent、Workflow、Skills 与 MCP

DSH 的子 Agent 不只是一次性 task。它区分 one-shot 与 continuable child；continuable child 的 durable Session 与进程内 Activation 分离，followup 根据 activation 的 running/waiting/cold 状态投递到同一 inbox，父子权限由真实 live Agent 与 `parentSession` 验证。fork provider 可继承父会话完成 turn 的前缀；spawn provider 从空上下文开始。Subagent 文档

Workflow 是模型写脚本来编排多个 child agent 的另一条路径；当前 worker-thread + `node:vm` 也明确不构成不可信脚本沙箱。它适合受信 orchestration，不适合作为多租户隔离层。

Skills 使用全局 + per-scope 分层 registry，并从 system/user/project 路径发现；Agent preset 可以改变技能和工具表面。MCP client 支持 stdio 与 Streamable HTTP，动态发现工具后以 `mcp__<server>__<tool>` 注册，崩溃后带退避重连；默认发行版不预装内存 MCP 服务。MCP 指南

这种统一 registry 的优点是 MCP、内建工具和插件工具共享政策/日志/呈现；缺点是第三方 MCP 返回内容直接扩大间接 prompt injection 面，工具 schema 与外部服务生命周期也会让每个会话的可复现性更难保证。

8. 安全：正确的威胁模型

8.1 已有控制

- 默认 `workspace-write + ask`，支持 `read-only` 与 `danger-full-access`。
- Linux 优先 bubblewrap，其次 Landlock；macOS 使用 Seatbelt；Windows 使用 ACL restricted token。不可用时 fail closed，不静默退回无沙箱。
- 每次 capability call 携带完整 policy；cwd 会先按 filesystem semantics canonicalize，减少 symlink/`..` 边界混淆。
- 当前 Web profile 默认 loopback；启动 URL 含随机 token，交换成绑定 authority 的签名 cookie；CLI 目前拒绝 `0.0.0.0`。Web 安全现状

8.2 必须明确的边界

- SandboxMode 只描述文件副作用。网络访问和进程可见性不在这套 vocabulary 中；same-world sandbox 仍共享宿主内核与文件系统。沙箱文档
- Windows ACL 与旧 Landlock ABI 可能只能 partial enforcement；Seatbelt 依赖已弃用的 `sandbox-exec`。
- PTC `run_code` 和 Workflow worker 不是 OS 沙箱；插件代码更是宿主进程受信代码。
- API key 存在本地 `.credentials.yaml`；UI 只拿到 redacted descriptor，但本机进程权限与备份策略仍是边界。Provider 配置
- 官方明确表示未完成安全审计，并建议使用一次性 VM、容器或专用环境。官方 Safety

8.3 间接 Prompt Injection

腾讯朱雀实验室的 A.I.G 评估在一个 DSH revision、一个模型 backend、一个 persona 和未启用 prompt hardening 的基线下执行 14,560 次控制实验，覆盖 16 个间接内容通道、35 个目标和 12 种攻击。最强观测攻击成功率分别达到 17.0%（LLM judge，文本 fake-completion）、25.5%（rule judge，文件 hidden Unicode）与 16.0%（rule judge，Skills 文件通道）。这些数字不能外推到所有模型/配置，却足以证明：仅靠模型提示词不能成为敏感 action 的安全边界。研究论文

8.4 旧审计报告如何解读

2026-08 中旬社区报告针对 `0.1.0-rc.5/rc.6` 提到 Web RPC 无认证等问题。当前 alpha.2 源码已经出现 token/cookie 认证和拒绝全接口绑定，因此旧结论不能直接视为当前漏洞。另一方面，PTC worker 非 OS 隔离仍是当前官方文档明确承认的设计边界。结论应按 commit 固定，而不是按“DSH”品牌整体判断。

9. 性能：有哪些证据，哪些只是推断

可以从源码确认的机制：增量 message projection、固定窗口批量持久化、projection cache、tool rolling pool、PTC subcall pool、tool result pruning、compaction、输出 spill、长会话 UI 增量处理。最新 release 也声明改善长历史与密集实时消息处理。v0.1.2-alpha.2 release

不能从公开材料确认的：相对 Codex/Claude Code/OpenCode 的 SWE-bench 成功率、任务时延、token 成本、内存曲线、并发 session 上限、插件/HMR 抖动和多小时任务稳定性。DSH 是 harness，不同模型、prompt、preset、provider、工具模式会主导结果；没有固定实验矩阵，单一 benchmark 几乎没有解释力。

因此合理的性能结论是：架构为长会话、并行工具和重放做了认真优化，但尚无公开证据证明它在真实 coding tasks 上整体更快或更省。

10. 与相邻方案的定位

方案	核心抽象	最适合	相对 DSH
DSH	插件树 + event-sourced agent runtime	自定义 harness、可重组 Agent 产品、运行轨迹研究	可组合性最激进，成熟度最低，TypeScript/Cordis 心智成本高
OpenAI Codex OSS	Rust 核心的 coding agent、CLI/App/SDK 与 OS sandbox	直接完成软件工程任务、成熟交互和本地安全策略	产品与执行路径更成熟；不是“所有能力可替换”的插件内核。Codex repo
LangGraph	显式 graph/state/checkpoint 的 orchestration runtime	业务工作流、长运行 durable graph、人机中断	控制流更显式、偏应用编排；DSH 更像完整 agent 产品与插件发行版。LangGraph

DSH 与 Codex/Claude Code 的竞争不只在“谁写代码更好”，而在“谁拥有 Harness 层的可塑性”。与 LangGraph 的差异则是：LangGraph 让开发者画出控制图；DSH 让模型驱动默认 loop，再通过插件和事件重组运行时。

11. 采用建议

立即值得试

- Agent runtime / plugin platform 团队，需要替换模型、工具、存储、沙箱、子 Agent provider。
- 研究不同 prompt/tool/preset 对轨迹的影响，需要完整可回放日志。
- 内部 coding agent 原型，能接受 breaking changes，并愿意将整个 Harness 放进 disposable container/VM。
- 希望构建 DSH 插件生态或把 MCP、Skills、schedule、workflow 组合成定制产品。

暂缓生产

- 要处理任意互联网内容、恶意仓库或未知第三方插件，又不能提供外层容器隔离。
- 对会话格式长期兼容、版本迁移、审计认证、企业 SLA 有硬性要求。
- 团队只需要一个开箱即用 coding agent，不准备投入 Cordis/TypeScript 平台工程。
- 需要证明相对现有 agent 的任务成功率或成本收益，但无法自己做 A/B benchmark。

推荐的 30 天验证

1. 固定 commit、Node 与 pnpm 版本；不要跟随 master 漂移。
2. 在容器/VM 内部署，Web 保持 loopback，经 SSH tunnel 访问；禁用不需要的网络与凭据。
3. 先用 `native` + `workspace-write + ask`；PTC 仅在单独隔离环境开启。
4. 选择 30-50 个真实仓库任务，固定模型，对比当前 agent：成功率、模型轮数、工具次数、token、wall time、人工批准次数、失败恢复。
5. 注入恶意 README、网页、Skills 与 MCP result，验证敏感工具必须经过确定性 policy，而不是只依赖 system prompt。
6. 进行三次升级/降级演练，验证旧 session、插件 preset 与凭据备份；把 session format break 当作预期事件。
7. 只有在安全边界、任务收益和升级成本三项同时过线后，再进入有限生产试点。

12. 最终判断

DSH 的野心是把 Agent Harness 从“一段写死的循环”升级为可重组、可回放、可热替换的软件系统。Cordis + event-sourced Session + capability seams 的组合在开源 agent 项目中非常少见，设计质量也明显高于普通 demo。

它的主要风险恰恰来自同一野心：极端插件化带来认知与组合复杂度；alpha 期的 API/session 兼容性尚未稳定；宿主插件、PTC 与 workflow 的信任边界容易被“看起来有 worker/sandbox”误解；公开端到端性能证据不足。

一句话结论：DSH 值得被当作下一代 Agent runtime 架构样本深入研究，也值得做隔离环境中的平台原型；但截至 0.1.2-alpha.2，不应被当作无需额外安全工程即可上线的生产 coding agent。

资料与证据账本

- DeepSeek, “DeepSeek Harness developer preview: Everything is a plugin”, 2026-08.
- deepseek-ai, `deepseek-harness` repository and README, accessed 2026-08-31.
- deepseek-ai, release `dsh-v0.1.2-alpha.2`, 2026-08-30.
- Shi, Zhang, Cui, “A Programming Paradigm for Spatiotemporal Composability”, 2026-08-26.
- deepseek-ai, Architecture, Session, Persistence, Tools, Sandbox, Subagent and Code Runtime documentation, accessed 2026-08-31.
- deepseek-ai, Safety notice, accessed 2026-08-31.
- Ying et al., “Security Assessment of DeepSeek Harness with A.I.G”, 2026-08-17.
- OpenAI, Codex open-source repository, accessed 2026-08-31.
- LangChain, LangGraph documentation, accessed 2026-08-31.

研究限制

- 研究基于公开材料和静态源码审阅，没有运行真实 DeepSeek API，也没有执行破坏性安全 PoC。
- GitHub Discussions 中的社区漏洞只作为发现线索；未在当前 commit 复现的，不作为当前漏洞结论。
- 未发现可解释 DSH 整体能力的官方或独立端到端 benchmark；报告将此明确列为证据缺口，而非以架构推断替代数据。
- 项目处于快速迭代期；结论仅对写明的 commit 与日期负责。